

K-WaterGuard AI Student Technical Manual

[Readable classroom PDF edition](#)

Prepared for students learning agentic AI, data automation, dashboards, and deployment.

Table of Contents

1. About This Manual
2. Learning Goals
3. System Overview
4. Architecture Diagram
5. Main Files
6. Required Software
7. Python Dependencies
8. Step 1: Create The Project Folder
9. Step 2: Create A Virtual Environment
10. Step 3: Install Dependencies
11. Step 4: Understand The Main Code Structure
12. The `Config` Class
13. Data Source Configuration
14. Water-Quality Parameters
15. Alert Rules
16. The Collector
17. API Request Pattern
18. Coordinate Conversion
19. The Data Manager
20. Data Output Layout
21. The Plot Generator
22. The Dashboard Generator
23. Suggested Dashboard Layout
24. The Main Agent
25. Running The Agent Manually
26. Running With A Batch File
27. Windows Task Scheduler Setup
28. GitHub Pages Deployment
29. Google Sites Embedding
30. Optional Chatbot
31. Simple Chatbot Backend Example
32. Testing Checklist
33. Common Problems

- 34. Student Exercise
- 35. Mini Project Ideas
- 36. Final Summary

About This Manual

This manual explains how to build, run, understand, and deploy K-WaterGuard AI. It is written for students who are learning how an agentic AI/data automation system is designed from real code.

K-WaterGuard AI is a Python-based water-quality monitoring agent for South Korea. It collects data from a public Korean water-quality API, stores the data in CSV files, creates plots and maps, builds an HTML dashboard, and prepares the dashboard for GitHub Pages or Google Sites.

Learning Goals

After completing this manual, students should be able to:

- Explain the architecture of a data-collection agent.
- Set up a Python virtual environment.
- Install and manage project dependencies.
- Understand how API data is collected and parsed.
- Understand how water-quality records are saved and archived.
- Generate charts and spatial maps from collected data.
- Build a static HTML dashboard.
- Deploy the dashboard with GitHub Pages.
- Connect an optional chatbot backend safely.

System Overview

K-WaterGuard AI follows a simple agent workflow:

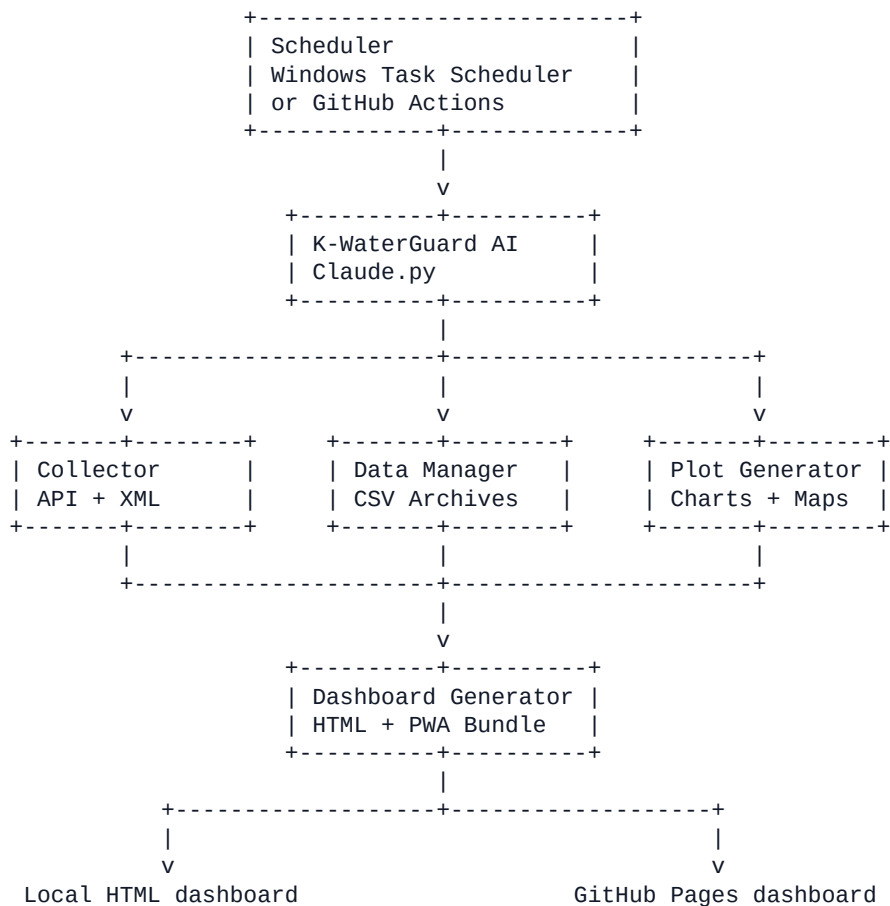
```
Collect data
-> Clean and enrich records
-> Save CSV files
-> Generate plots and maps
-> Build dashboard
-> Publish dashboard
```

The project is currently implemented mainly in one file:

`Claude.py`

This single file contains configuration, data collection, data storage, plotting, dashboard generation, and the main agent runner.

Architecture Diagram



Main Files

AI_Agent_Try/
Claude.py
requirements.txt
run_water_agent.bat
Kwater.png
KwGAI logo.png
cover.png
ctprvn.shp
ctprvn.shx
ctprvn.dbf
.github/workflows/update-dashboard-pages.yml
water_quality_data/

Important file meanings:

- Claude.py: Main agent source code.
- requirements.txt: Python package list.
- run_water_agent.bat: Windows script for running the agent.
- ctprvn.shp, ctprvn.shx, ctprvn.dbf: South Korea map shapefile components.
- water_quality_data/: Generated data, plots, dashboards, and logs.
- .github/workflows/update-dashboard-pages.yml: GitHub Actions workflow for automatic dashboard publishing.

Required Software

Install these before building the system:

- Python 3.11 or newer.
- Visual Studio Code or another code editor.
- Git, if publishing to GitHub.
- A modern web browser.
- Optional: GitHub account for GitHub Pages deployment.

Python Dependencies

The project uses these Python packages:

```
numpy
requests
pandas
matplotlib
seaborn
apscheduler
pyproj
reverse_geocoder
```

Package purposes:

- requests: Sends HTTP requests to the water-quality API.
- pandas: Stores and processes tabular data.
- numpy: Supports numeric operations.
- matplotlib: Creates charts and maps.
- seaborn: Creates statistical visualizations.
- apscheduler: Runs scheduled jobs inside Python.
- pyproj: Converts coordinates between map systems.
- reverse_geocoder: Helps infer location names from coordinates.

Step 1: Create The Project Folder

Create a project folder:

```
mkdir C:\Users\USER\Desktop\AI_Agent_Try
cd C:\Users\USER\Desktop\AI_Agent_Try
```

Copy the project files into this folder.

Step 2: Create A Virtual Environment

Run:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

If PowerShell blocks activation, run:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
.\.venv\Scripts\Activate.ps1
```

Step 3: Install Dependencies

Install all packages:

```
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

Check that the environment works:

```
python -c "import pandas, requests, matplotlib; print('Environment OK')"
```

Step 4: Understand The Main Code Structure

Claude.py contains these major parts:

```
ensure_dependencies()
Config
setup_environment()
WaterQualityCollector
DataManager
PlotGenerator
DashboardGenerator
WaterQualityAgent
main()
```

Each part has a specific job.

The `Config` Class

The Config class stores global settings.

Important examples:

```
class Config:
    DATA_DIR = Path(os.environ.get(
        "WATER_QUALITY_DATA_DIR",
        str(Path.home() / "water_quality_data")
    ))

    CSV_FILE = DATA_DIR / "water_quality_records.csv"
    PLOTS_DIR = DATA_DIR / "plots"
    DASHBOARD_FILE = DATA_DIR / "dashboard.html"
    SITE_DASHBOARD_DIR = DATA_DIR / "google_site_dashboard"
```

This means the agent can save output either in the default user folder or in a custom folder using an environment variable.

Data Source Configuration

The water-quality API is configured in Config:

```
WATER_API_URL = "http://apis.data.go.kr/1480523/WaterqualityServices/getIvstgWFS"
WFS_SRS_NAME = "EPSG:5179"
WFS_MAX_FEATURES = 5000
WFS_RESULT_TYPE = "results"
```

The API returns station features. The agent parses those features and converts them into rows of water-quality data.

Water-Quality Parameters

The agent tracks these parameters:

pH
DO
BOD
COD
SS
TN
TP
temperature
EC
Turbidity
Chlorophyll_a
Fecal_Coliiform
E_coli
Alkalinity
Hardness
Ammonia_N
Nitrate_N
Phosphate_P

Students should know:

- pH measures acidity or alkalinity.
- DO means dissolved oxygen.
- BOD means biochemical oxygen demand.
- COD means chemical oxygen demand.
- TN means total nitrogen.
- TP means total phosphorus.

Alert Rules

Alert rules are stored in `Config.WATER_QUALITY_ALERT_RULES`.

Example:

```
"pH": {  
  "min": 6.5,  
  "max": 8.5,  
  "unit": "",  
  "severity": "critical",  
  "basis": "Korean river/lake living-environment pH range"  
}
```

The logic is:

If the value is below min -> create alert.
If the value is above max -> create alert.
If pH is outside 6.5 to 8.5 -> create alert.

The Collector

The collector is responsible for getting data from the API.

Class:

```
class WaterQualityCollector:
```


Main method:

```
def fetch_data(self):
    data = self._fetch_api_data()
    if data:
        return data
    return fallback_data
```

The collector does four important things:

- Sends a request to the API.
- Parses XML data.
- Extracts station names and station codes.
- Extracts water-quality parameters.

API Request Pattern

The API request uses `requests.get`:

```
params = {
    "serviceKey": Config.SERVICE_KEY,
    "srsName": Config.WFS_SRS_NAME,
    "maxFeatures": Config.WFS_MAX_FEATURES,
    "resultType": Config.WFS_RESULT_TYPE,
}

response = requests.get(
    Config.WATER_API_URL,
    params=params,
    headers=self.headers,
    timeout=60
)
response.raise_for_status()
```

`raise_for_status()` makes Python raise an error if the API response fails.

Coordinate Conversion

The API uses Korean projected coordinates:

EPSG:5179

Web maps and dashboards usually need:

EPSG:4326
latitude and longitude

The code uses `pyproj`:

```
Transformer.from_crs("EPSG:5179", "EPSG:4326", always_xy=True)
```

This allows stations to be placed correctly on maps.

The Data Manager

Class:

```
class DataManager:
```

Main job:

Receive new records
-> Convert to pandas DataFrame
-> Clean location columns
-> Save master CSV
-> Save daily CSV
-> Save run archive
-> Save location exports

Important output:

water_quality_data/water_quality_records.csv

This is the master dataset.

Data Output Layout

After a run, the folder should look like this:

```
water_quality_data/  
  water_quality_records.csv  
  dashboard.html  
  agent_log.txt  
  plots/  
  location_exports/  
  run_archives/  
  daily_outputs/  
    YYYY-MM-DD/  
      data/  
      plots/  
      runs/  
  google_site_dashboard/
```

This layout helps separate:

- Long-term records.
- Daily outputs.
- Charts.
- Run archives.
- Website-ready files.

The Plot Generator

Class:

```
class PlotGenerator:
```

The plot generator creates:

- Quality summary chart.
- Regional comparison chart.
- Heatmap.
- Parameter distribution charts.
- Alert hotspot matrix.
- Station coverage map.
- Spatial parameter maps.

Important method:

```
def generate_all_plots(self):
    df = DataManager().get_latest_data(days=7)
    self._plot_quality_summary(df)
    self._plot_station_coverage_map(df)
    self._plot_parameter_maps(df)
```

Generated images are saved as .png files.

The Dashboard Generator

Class:

```
class DashboardGenerator:
```

The dashboard generator creates:

- Local dashboard HTML.
- Workspace dashboard copy.
- GitHub Pages bundle.
- Progressive Web App files.
- Optional chatbot interface.

Important outputs:

```
water_quality_data/dashboard.html
Claude_dashboard.html
water_quality_data/google_site_dashboard/index.html
```

Suggested Dashboard Layout

```
+-----+
| Header: Logo, title, latest date, station count |
+-----+
| Side Panel | Main Dashboard |
| - Map summary | - Parameter cards |
| - Province stats | - Alert table |
| - History links | - Charts |
| - Chatbot button | - Spatial maps |
+-----+
```

Good dashboard design principles:

- Put the most important status at the top.
- Use alert colors only where needed.
- Keep charts grouped by purpose.
- Make CSV downloads easy to find.
- Do not expose private API keys in the page.

The Main Agent

Class:

```
class WaterQualityAgent:
```

The agent connects all components:

```
class WaterQualityAgent:
    def __init__(self):
        self.collector = WaterQualityCollector()
        self.data_manager = DataManager()
        self.plot_generator = PlotGenerator()
        self.dashboard_generator = DashboardGenerator()

    def execute_cycle(self):
        data = self.collector.fetch_data()
        if data:
            self.data_manager.save_data(data)
            self.plot_generator.generate_all_plots()
            self.dashboard_generator.generate()
```

This is the central agentic loop.

Running The Agent Manually

Run:

```
.\.venv\Scripts\python.exe Claude.py
```

Expected result:

1. Data is collected.
2. CSV files are saved.
3. Plots are generated.
4. Dashboard HTML is created.
5. Google Sites dashboard bundle is created.

Running With A Batch File

The batch file can run the agent:

```
@echo off
cd /d "C:\Users\USER\Desktop\AI_Agent_Try"
".venv\Scripts\python.exe" Claude.py
```

Save it as:

```
run_water_agent.bat
```

Windows Task Scheduler Setup

Use Task Scheduler when you want the agent to run automatically on your computer.

Recommended settings:

```
Task name: Water Quality Agent
Trigger: Daily or At log on
Action: run_water_agent.bat
Start in: C:\Users\USER\Desktop\AI_Agent_Try
```

Important:

- Remove pause from the batch file for automatic runs.
- Use the virtual environment Python path.
- Check agent_log.txt after scheduled runs.

GitHub Pages Deployment

The project includes a GitHub Actions workflow:

`.github/workflows/update-dashboard-pages.yml`

It does this:

```
Checkout repository
-> Set up Python
-> Install requirements
-> Run Claude.py
-> Upload generated dashboard
-> Deploy to GitHub Pages
```

Enable GitHub Pages:

```
GitHub repository
-> Settings
-> Pages
-> Source: GitHub Actions
```

Expected dashboard URL:

`https://YOUR_GITHUB_USERNAME.github.io/k-waterguard-dashboard/`

Google Sites Embedding

Google Sites cannot show a local file from your computer. It needs a public HTTPS URL.

Correct flow:

```
Claude.py
-> GitHub Pages dashboard
-> Google Sites embed
```

Steps:

- Open Google Sites editor.
- Click Insert.
- Click Embed.
- Paste the GitHub Pages URL.
- Resize the embedded frame.
- Publish the Google Site.

Optional Chatbot

The dashboard has optional chatbot support.

Important security rule:

Never put an OpenAI API key inside dashboard HTML or frontend JavaScript.

Correct chatbot flow:

```
Student/user question
-> Dashboard frontend
-> Backend API
-> OpenAI API
```

- > Backend response
- > Dashboard answer

The dashboard expects the backend to return:

```
{
  "answer": "The chatbot answer goes here."
}
```

Environment variable:

```
$env:CHATBOT_API_URL = "https://your-backend-domain.com/api/chat"
```

Simple Chatbot Backend Example

This is a simplified FastAPI backend pattern:

```
import os
from fastapi import FastAPI
from pydantic import BaseModel
from openai import OpenAI

app = FastAPI()
client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])

class ChatRequest(BaseModel):
    question: str

@app.post("/api/chat")
def chat(req: ChatRequest):
    response = client.responses.create(
        model="gpt-4.1-mini",
        input=f"Answer this water-quality question: {req.question}"
    )
    return {"answer": response.output_text}
```

This backend must be hosted on a server, not inside GitHub Pages.

Testing Checklist

Before giving the project to students, test:

```
.\.venv\Scripts\python.exe -m py_compile Claude.py
.\.venv\Scripts\python.exe Claude.py
```

Check these files:

```
water_quality_data/water_quality_records.csv
water_quality_data/dashboard.html
water_quality_data/google_site_dashboard/index.html
Claude_dashboard.html
```

Check plots:

```
Get-ChildItem water_quality_data -Recurse -Filter *.png
```

Common Problems

Problem:

Dashboard opens but images are missing.

Possible solution:

Regenerate the dashboard and confirm PNG files exist in google_site_dashboard.

Problem:

API returns no records.

Possible solution:

Check internet connection, API key, API URL, and API response status.

Problem:

Map does not render.

Possible solution:

Confirm ctpvsn.shp, ctpvsn.shx, and ctpvsn.dbf are all present.

Problem:

Chatbot does not answer.

Possible solution:

Confirm CHATBOT_API_URL is set and the backend returns JSON with an answer field.

Student Exercise

Ask students to complete these tasks:

- Run the agent locally.
- Find the generated master CSV.
- Open the dashboard.
- Identify the plot generation code.
- Change one alert threshold.
- Regenerate the dashboard.
- Explain what changed.
- Draw their own architecture diagram.

Mini Project Ideas

Students can extend K-WaterGuard AI by adding:

- A station search box.
- A date filter.
- A downloadable PDF report.
- A database instead of CSV storage.
- More water-quality standards.
- A chatbot that answers questions from the latest CSV.
- Email alerts for critical water-quality changes.

Final Summary

K-WaterGuard AI is a complete agentic data system. It combines data collection, data engineering, visualization, dashboard generation, automation, and optional AI chatbot support.

The most important idea is the agent cycle:

Collect -> Store -> Analyze -> Visualize -> Publish

This pattern can be reused for many real-world monitoring systems, including air quality, weather, energy, traffic, agriculture, and public health dashboards.